



ENSEEIH – École Nationale Supérieure d'Électrotechnique,
d'Électronique, d'Informatique, d'Hydraulique et des
Télécommunications

Département Sciences du Numérique (SN) – Première année

SYSTÈMES D'EXPLOITATION CENTRALISÉS

Projet Minishell : Test et validation des étapes du développement et synthèses des travaux pratiques

Auteur :

M. Lucas MOUSSY
Étudiant ingénieur – SN1

Encadrant :

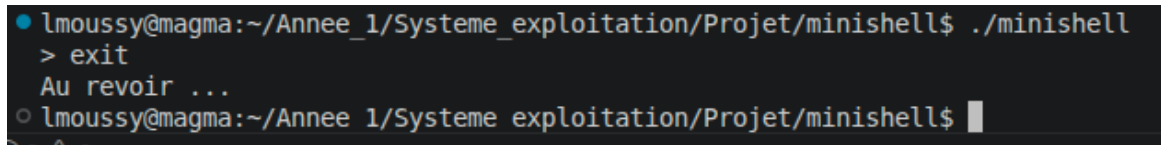
M. Antonin VERDIER

Table des matières

1	TP 1 – Gestion des Processus et Fondations du Minishell	2
1.1	Étape 1 : Testez le programme	2
1.2	Étape 2 : Lancement d'une commande	2
1.3	Étape 3 : Enchaînement séquentiel des commandes	3
1.4	Étape 4 : Lancement de commandes en tâche de fond	3
2	TP 2 – Gestion Asynchrone et Signaux	4
2.1	Étape 5 : Traitement du signal SIGCHLD	4
2.2	Étape 6 : Utilisation de SIGCHLD pour traiter la terminaison des processus fils	4
2.3	Étape 7 : Attendre un signal : pause	4
2.4	Étape 8 : Suspension et reprise des processus en arrière-plan	5
2.5	Étape 9 : Affichage d'un message indiquant le signal reçu	5
3	TP 3 – Contrôle Terminal et Masques de Signaux	6
3.1	Étape 10 : Test de la frappe au clavier de ctrl-C et ctrl-Z	6
3.2	Étape 11 : Gestion de la frappe au clavier de ctrl-C et ctrl-Z	6
3.3	Étape 12 : Détacher les processus fils en arrière-plan	6
4	TP 4 – Système de Fichiers, Redirections et Commandes Internes	7
4.1	Étape 13 : Redirections	7
4.2	Étape 14 : Déplacement dans l'arborescence	7
4.3	Étape 15 : Afficher le contenu d'un répertoire	8
5	TP 5 – Communication Inter-processus via les Tubes	9
5.1	Étape 16 : Tubes simples	9
5.2	Étape 17 : Pipelines	9

1 TP 1 – Gestion des Processus et Fondations du Minishell

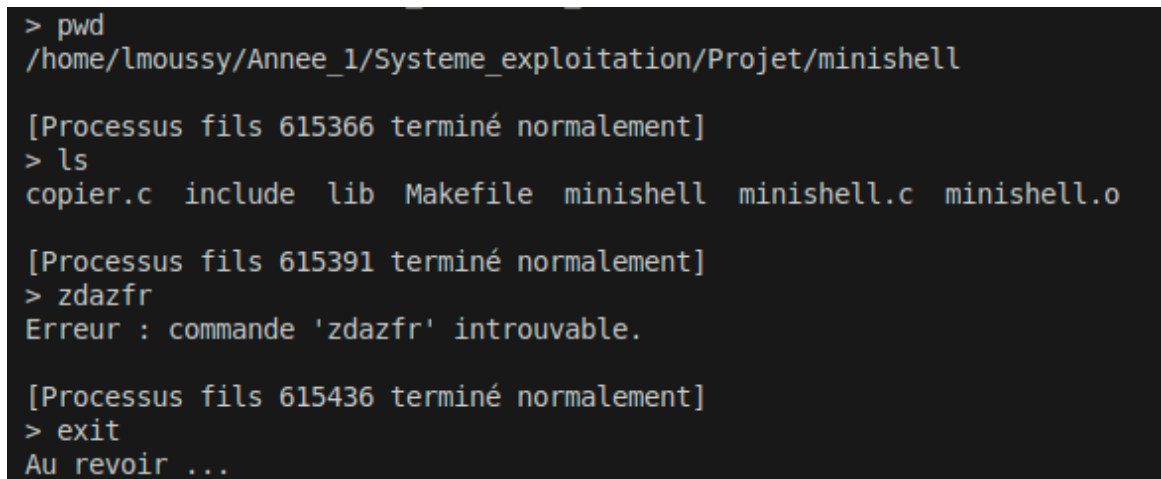
1.1 Étape 1 : Testez le programme



```
• lmoussy@magma:~/Annee_1/Systeme_exploitation/Projet/minishell$ ./minishell
> exit
Au revoir ...
○ lmoussy@magma:~/Annee_1/Systeme_exploitation/Projet/minishell$
```

FIGURE 1 – Étape 1

1.2 Étape 2 : Lancement d'une commande



```
> pwd
/home/lmoussy/Annee_1/Systeme_exploitation/Projet/minishell

[Processus fils 615366 terminé normalement]
> ls
copier.c include lib Makefile minishell minishell.c minishell.o

[Processus fils 615391 terminé normalement]
> zdazfr
Erreur : commande 'zdazfr' introuvable.

[Processus fils 615436 terminé normalement]
> exit
Au revoir ...
```

FIGURE 2 – Étape 2

Exécution de différentes commandes.

1.3 Étape 3 : Enchaînement séquentiel des commandes

```
> date
mer. 20 mai 2026 17:04:55 CEST

[Processus fils 624898 terminé normalement]
> sleep 100
ls
pwd

[Processus fils 624952 terminé normalement]
> ls
copier.c include lib Makefile minishell minishell.c minishell.o

[Processus fils 625899 terminé normalement]
> pwd
/home/lmoussy/Annee_1/Systeme_exploitation/Projet/minishell

[Processus fils 625900 terminé normalement]
> date
mer. 20 mai 2026 17:06:57 CEST

[Processus fils 626078 terminé normalement]
> █
```

FIGURE 3 – Étape 3

On observe que lorsque la commande `sleep` est lancée notre terminal attend que celle-ci est fini de s'exécuter avant d'exécuter nos deux autres commandes qui sont `ls` et `pwd`.

1.4 Étape 4 : Lancement de commandes en tâche de fond

```
> sleep 10 &
> pwd
/home/lmoussy/Annee_1/Systeme_exploitation/Projet/minishell

[Processus fils 621861 terminé normalement]
>
[Processus fils 621818 terminé normalement]
> exit
Au revoir ...
```

FIGURE 4 – Étape 4

La commande `sleep 10` s'exécute bien en fond et on peut en même temps lancer la commande `pwd` avant que celle-ci se termine.

2 TP 2 – Gestion Asynchrone et Signaux

2.1 Étape 5 : Traitement du signal SIGCHLD

```
> sleep 2 &  
>  
[Processus fils 630985 terminé normalement]  
█
```

FIGURE 5 – Étape 5

Le père intercepte de manière asynchrone la fin d'un processus en tâche de fond grâce au signal SIGCHLD. Après 2 secondes d'attentes le message de mort apparaît tout seul.

2.2 Étape 6 : Utilisation de SIGCHLD pour traiter la terminaison des processus fils

```
> sleep 10 &  
> sleep 10 &  
> sleep 10 &  
>  
[Processus fils 633750 terminé normalement]  
  
[Processus fils 633788 terminé normalement]  
  
[Processus fils 633822 terminé normalement]  
> █
```

FIGURE 6 – Étape 6

Notre waitpid intercepte le premier signal et nettoyer les 3 zombies d'un seul affichant ainsi les 3 messages de terminaison à la suite.

2.3 Étape 7 : Attendre un signal : pause

```
> sleep 1000  
█
```

```
lmoussy 636610 0.0 0.0 8920 3172 pts/0 S+ 17:25 0:00 ./minishell
```

FIGURE 7 – Étape 7

Lorsque que notre commande sleep 1000 est lancée, celle-ci tourne en avant plan. On observe bien que le père consomme 0 % du CPU et se met en attente passive.

2.4 Étape 8 : Suspension et reprise des processus en arrière-plan

```
> sleep 100 &
>
[Processus fils 642531 suspendu]
```

```
lmoussy 642531 642471 0 17:34 pts/0 00:00:00 sleep 100
lmoussy 642849 636352 0 17:34 pts/1 00:00:00 grep sleep
lmoussy@chevesne:~/Annee_1/Systeme_exploitation$ kill -STOP 642531
```

FIGURE 8 – Étape 8

Utilisation de la commande kill pour suspendre un processus en arrière-plan.

2.5 Étape 9 : Affichage d'un message indiquant le signal reçu

```
[Processus fils 642531 terminé normalement]
```

```
lmoussy@chevesne:~/Annee_1/Systeme_exploitation$ kill -CONT 642531
```

FIGURE 9 – Étape 9

Notre shell affiche bien de manière dynamique l'action exacte qui se produit sur nos tâche de fond ici sur le sleep de l'étape 8 qu'on a fait reprendre.

3 TP 3 – Contrôle Terminal et Masques de Signaux

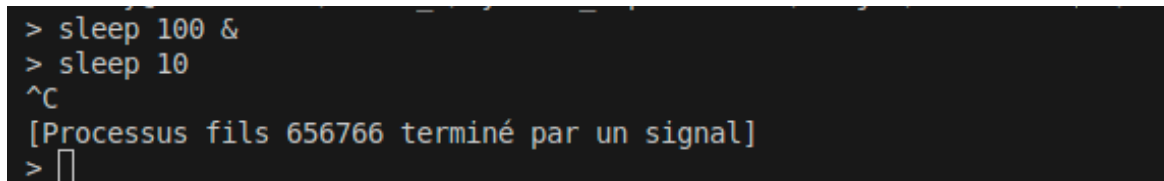
3.1 Étape 10 : Test de la frappe au clavier de ctrl-C et ctrl-Z

On ne peut le montrer le problème du ctrl-C et ctrl-Z étant corrigé dans la suite du projet.

3.2 Étape 11 : Gestion de la frappe au clavier de ctrl-C et ctrl-Z

Le ctrl-C n'arrête plus notre minishell. Lorsqu'on le tape rien ne se passe lorsqu'aucun processus ne tourne en premier plan. De même pour le ctrl-Z. Corrige le problème de l'étape 10.

3.3 Étape 12 : Détacher les processus fils en arrière-plan

A terminal window with a dark background and light-colored text. The text shows a user entering two background sleep commands, pressing Ctrl-C, and receiving a message that the child process has been terminated by a signal.

```
> sleep 100 &  
> sleep 10  
^C  
[Processus fils 656766 terminé par un signal]  
> █
```

FIGURE 10 – Étape 12

Ctrl-C arrête bien notre processus en avant-plan (sleep 10).

4 TP 4 – Système de Fichiers, Redirections et Commandes Internes

4.1 Étape 13 : Redirections

```
lmoussy@sunfire:~/Annee_1/Systeme_exploitation/Projet/minishell$ echo "Test de redirection reussi !" > fichier_entree.txt
lmoussy@sunfire:~/Annee_1/Systeme_exploitation/Projet/minishell$ ./minishell
> cat < fichier_entree.txt > fichier_sortie.txt

[Processus fils 3696206 terminé normalement]
> exit
Au revoir ...
lmoussy@sunfire:~/Annee_1/Systeme_exploitation/Projet/minishell$ cat fichier_sortie.txt
Test de redirection reussi !
lmoussy@sunfire:~/Annee_1/Systeme_exploitation/Projet/minishell$
```

FIGURE 11 – Étape 13

Cat est en mesure d'ouvrir fichier_entree.txt validant que l'entrée standard a bien été modifiée par le fichier txt.

4.2 Étape 14 : Déplacement dans l'arborescence

```
> pwd
/home/lmoussy/Annee_1/Systeme_exploitation/Projet/minishell

[Processus fils 3701179 terminé normalement]
> cd include/
> pwd
/home/lmoussy/Annee_1/Systeme_exploitation/Projet/minishell/include

[Processus fils 3701458 terminé normalement]
> cd..
Erreur : commande 'cd..' introuvable.

[Processus fils 3701533 terminé normalement]
> cd ..
> pwd
/home/lmoussy/Annee_1/Systeme_exploitation/Projet/minishell

[Processus fils 3701717 terminé normalement]
> cd
> pwd
/home/lmoussy

[Processus fils 3701771 terminé normalement]
>
```

FIGURE 12 – Étape 14 : Exécution de la commande interne cd et gestion de HOME

Illustration du bon fonctionnement de l'arborescence.

4.3 Étape 15 : Afficher le contenu d'un répertoire

```
> rep
.
..
lib
minishell.c
include
fichier_sortie.txt
Makefile
minishell.o
copier.c
fichier_entree.txt
minishell
> rep minishell
Erreur opendir: Not
> rep include/
.
..
readcmd.h
> 
```

FIGURE 13 – Étape 15

Illustration du bon fonctionnement de rep.

5 TP 5 – Communication Inter-processus via les Tubes

5.1 Étape 16 : Tubes simples

```
> ls
copier.c  fichier_entree.txt  fichier_sortie.txt  include  lib  Makefile  minishell  minishell.c  minishell.o

[Processus fils 3630324 terminé normalement]
> ls | wc -l

[Processus fils 3630505 terminé normalement]
9

[Processus fils 3630506 terminé normalement]
> 
```

FIGURE 14 – Étape 16

Le résultat 9 obtenu nous permet de justifier que le `ls` a bien été redirigé vers l'entrée du tube et `wc` a bien été redirigée vers ma sortie du tube.

5.2 Étape 17 : Pipelines

```
> cat minishell.c | grep fork | wc -l
3

[Processus fils 3632060 terminé normalement]
[Processus fils 3632061 terminé normalement]
[Processus fils 3632062 terminé normalement]
> 
```

FIGURE 15 – Étape 17

Notre premier tube connecte la sortie de `cat` à l'entrée de `wc`. Notre `wc -l` reçoit bien les données filtrées et affiche le 3 le compte finalce qui montre que notre variable temporaire de stockage du descripteur de lecture précédent passe correctement le relais de fils en fils.